Chapter 4 ▪ Networking

We begin this chapter by reviewing basic networking concepts, then move on to writing Java programs that connect to network services. We will show you how network clients and servers are implemented. Finally, you will see how to send e-mail from a Java program and how to harvest information from a web server.

4.1. Connecting to a Server

In the following sections, you will connect to a server, first by hand and with telnet, and then with a Java program.

4.1.1. Using Telnet

The telnet program is a great debugging tool for network programming. You should be able to launch it by typing `telnet` from a command shell.



Note: In Windows, you need to activate telnet. Go to the Control Panel, select Programs, click Turn Windows Features On or Off, and select the Telnet client checkbox. The Windows firewall also blocks quite a few network ports that we use in this chapter; you might need an administrator account to unblock them.

You may have used telnet to connect to a remote computer, but you can use it to communicate with other services provided by Internet hosts as well. Here is an example of what you can do. Type

```
telnet time-a.nist.gov 13
```

As **Figure 4.1** shows, you should get back a line like this:

```
57488 16-04-10 04:23:00 50 0 0 610.5 UTC(NIST) *
```

```
~$ telnet time-a.nist.gov 13
Trying 129.6.15.28...
Connected to time-a.nist.gov.
Escape character is '^]'.

60885 25-07-29 16:40:40 50 0 0  74.5 UTC(NIST) *
Connection closed by foreign host.
~$
```

Figure 4.1: Output of the “time of day” service

What is going on? You have connected to the “time of day” service that most UNIX machines constantly run. The particular server that you connected to is operated by the National Institute of Standards and Technology and gives the measurement of a Cesium atomic clock. (Of course, the reported time is not completely accurate due to network delays.)

By convention, the “time of day” service is always attached to “port” number 13.

Note: In network parlance, a port is not a physical device, but an abstraction facilitating communication between a server and a client (see Figure 4.2).

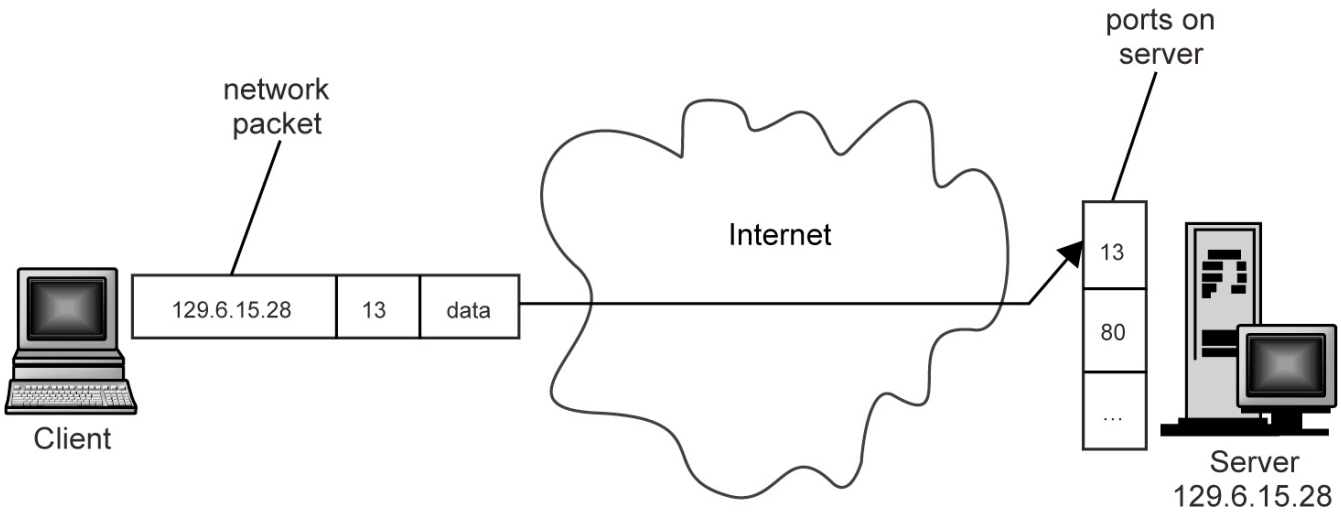


Figure 4.2: A client connecting to a server port

The server software is continuously running on the remote machine, waiting for any network traffic that wants to chat with port 13. When the operating system on the remote computer receives a network package that con-

tains a request to connect to port number 13, it wakes up the listening server process and establishes the connection. The connection stays up until it is terminated by one of the parties.

When you began the telnet session with `time-a.nist.gov` at port 13, a piece of network software knew enough to convert the string `"time-a.nist.gov"` to its correct Internet Protocol (IP) address, 129.6.15.28. The telnet software then sent a connection request to that address, asking for a connection to port 13. Once the connection was established, the remote program sent back a line of data and closed the connection. In general, of course, clients and servers engage in a more extensive dialog before one or the other closes the connection.

Here is another experiment along the same lines—but a bit more interesting. Type

```
telnet horstmann.com 80
```

Then type very carefully the following:

```
GET /index.html HTTP/1.1
Host: horstmann.com
blank line
```

That is, hit the Enter key twice at the end.

Figure 4.3 shows the response. It should look eerily familiar—you got a page of HTML-formatted text, namely Cay Horstmann's home page.



Figure 4.3: Using telnet to access an HTTP port

This is exactly the same process that your web browser goes through to get a web page. It uses HTTP to request web pages from servers. Of course, the browser displays the HTML code more nicely.



Note: The `Host` key/value pair is required when you connect to a web server that hosts multiple domains at the same IP address. You can omit it if the server hosts a single domain.

4.1.2. Connecting to a Server with Java

Our first network program in **Listing 4.1** will do the same thing we did using telnet—connect to a port and print out what it finds.

Listing 4.1 v2ch04/socket/SocketDemo.java

```
1  package v2ch04.socket;
2
3  import module java.base;
4
5  /**
6   * This program makes a socket connection to the atomic clock in Boulder, Colorado, and prints
7   * the time that the server sends.
8   */
9  class SocketDemo {
10     void main() throws Exception {
11         try (var s = new Socket("time-a.nist.gov", 13);
12                 var in = new Scanner(s.getInputStream())) {
13             while (in.hasNextLine()) {
14                 String line = in.nextLine();
15                 IO.println(line);
16             }
17         }
18     }
19 }
```

The key statements of this simple program are these:

```
var s = new Socket("time-a.nist.gov", 13);
InputStream inStream = s.getInputStream();
```

The first line opens a *socket*, which is a network software abstraction that enables communication out of and into this program. We pass the remote address and the port number to the socket constructor. If the connection fails, an `UnknownHostException` is thrown, which is a subclass of `IOException`.

The `getInputStream` method in `java.net.Socket` returns an `InputStream` object that you can use just like any other stream. The method throws an `IOException` if it is not possible to read from the socket. Once you have grabbed the stream, this program simply prints each input line to standard output. This process continues until the stream is finished and the server disconnects.

This program works only with very simple servers, such as a “time of day” service. In more complex networking programs, the client sends request data to the server, and the server might not immediately disconnect at the end of a response. You will see how to implement that behavior in several examples throughout this chapter.

The `Socket` class is pleasant and easy to use because the Java library hides the complexities of establishing a networking connection and sending data across it. The `java.net` package essentially gives you the same programming interface you would use to work with a file.



Note: In this book, we cover only the Transmission Control Protocol (TCP). The Java platform also supports the User Datagram Protocol (UDP), which can be used to send packets (also called *datagrams*) with much less overhead than TCP. The drawback is that packets need not be delivered in sequential order to the receiving application and can even be dropped altogether. It is up to the recipient to put the packets in order and to request retransmission of missing packets. UDP is well suited for applications in which missing packets can be tolerated—for example, for audio or video streams or continuous measurements.

`java.net.Socket` 1.0

- `Socket(String host, int port)`
  constructs a socket to connect to the given host and port.
- `InputStream getInputStream()`
- `OutputStream getOutputStream()`
  get the stream to read data from the socket or write data to the socket.

4.1.3. Socket Timeouts

Reading from a socket blocks until data are available. If the host is unreachable, your application waits for a long time and you are at the mercy of the underlying operating system to eventually time out.

You can decide what timeout value is reasonable for your particular application. Then, call the `setSoTimeout` method to set a timeout value (in milliseconds).

```
var s = new Socket(. . .);
s.setSoTimeout(10000); // time out after 10 seconds
```

If the timeout value has been set for a socket, all subsequent read operations throw a `SocketTimeoutException` when the timeout has been reached before the operation has completed its work. You can catch that exception and react to the timeout.

```
try {
    InputStream in = s.getInputStream(); // read from in
    . . .
}
catch (SocketTimeoutException e) {
    react to timeout
}
```

There is no timeout for write operations.

There is one additional timeout issue that you need to address. The constructor

```
Socket(String host, int port)
```

can block indefinitely until an initial connection to the host is established.

You can overcome this problem by first constructing an unconnected socket and then connecting it with a timeout:

```
var s = new Socket();
s.connect(new InetSocketAddress(host, port), timeout);
```

See **Section 4.2.4** for how to allow users to interrupt a socket connection at any time.

`java.net.Socket` 1.0

- `Socket()` 1.1
  creates a socket that has not yet been connected.
- `void connect(SocketAddress address)` 1.4
  connects this socket to the given address.
- `void connect(SocketAddress address, int timeoutInMilliseconds)` 1.4
  connects this socket to the given address, or throws a `SocketTimeoutException` if the time interval expired.
- `void setSoTimeout(int timeoutInMilliseconds)` 1.1
  sets the blocking time for read requests on this socket. If the timeout is reached when reading, a `SocketTimeoutException` is thrown.
- `boolean isConnected()` 1.4
- `boolean isClosed()` 1.4
  return `true` if the socket is connected or closed.

4.1.4. Internet Addresses

Usually, you don't have to worry too much about Internet addresses—the numerical host addresses that consist of 4 bytes (or, with IPv6, 16 bytes) such as 129.6.15.28. However, you can use the `InetAddress` class if you need to convert between host names and Internet addresses.

The `java.net` package supports IPv6 Internet addresses, provided the host operating system does.

The static `getByName` method returns an `InetAddress` object of a host. For example,

```
InetAddress address = InetAddress.getByName("time-a.nist.gov");
```

returns an `InetAddress` object that encapsulates the sequence of four bytes 129.6.15.28. You can access the bytes with the `getAddress` method.

```
byte[] addressBytes = address.getAddress();
```

Some host names with a lot of traffic correspond to multiple Internet addresses, to facilitate load balancing. For example, at the time of this writing, the host name `google.com` corresponds to twelve different Internet addresses. One of them is picked at random when the host is accessed. You can get all hosts with the `getAllByName` method.

```
InetAddress[] addresses = InetAddress.getAllByName(host);
```

Finally, you sometimes need the address of the local host. If you simply ask for the address of `localhost`, you always get the local loopback address 127.0.0.1, which cannot be used by others to connect to your computer.

Instead, use the static `getLocalHost` method to get the address of your local host.

```
InetAddress address = InetAddress.getLocalHost();
```

Listing 4.2 is a simple program that prints the Internet address of your local host if you do not specify any command-line arguments, or all Internet addresses of another host if you specify the host name on the command line, such as

```
java v2ch04.inetAddress.InetAddressDemo www.horstmann.com
```

Listing 4.2 v2ch04/inetAddress/InetAddressDemo.java

```
1  package v2ch04.inetAddress;
2
3  import module java.base;
4
5  /**
6   * This program demonstrates the InetAddress class. Supply a host name as command-line
7   * argument, or run without command-line arguments to see the address of the local host.
8   */
9  class InetAddressDemo {
10     void main(String[] args) throws Exception {
11         if (args.length > 0) {
12             String host = args[0];
13             InetAddress[] addresses = InetAddress.getAllByName(host);
14             for (InetAddress a : addresses)
15                 System.out.println(a);
16         }
17         else {
18             InetAddress localHostAddress = InetAddress.getLocalHost();
19             System.out.println(localHostAddress);
20         }
21     }
22 }
```

`java.net.InetAddress` **1.0**

- `static InetAddress getByName(String host)`
- `static InetAddress[] getAllByName(String host)`
  construct an `InetAddress`, or an array of all Internet addresses, for the given host name.
- `static InetAddress getLocalHost()`
  constructs an `InetAddress` for the local host.
- `byte[] getAddress()`
  returns an array of bytes that contains the numerical address.
- `String getHostAddress()`
  returns a string with decimal numbers, separated by periods, for example `"129.6.15.28"`.
- `String getHostName()`
  returns the host name.

4.2. Implementing Servers

Now that we have implemented a basic network client that receives data from the Internet, let's program a simple server that can send information to clients.

4.2.1. Server Sockets

A server program, when started, waits for a client to attach to its port. For our example program, we chose port number 8189, which is not used by any of the standard services. The `ServerSocket` class establishes a socket. In our case, the command

```
var s = new ServerSocket(8189);
```

establishes a server that monitors port 8189. The command

```
Socket incoming = s.accept();
```

tells the program to wait indefinitely until a client connects to that port. Once someone connects to this port by sending the correct request over the network, this method returns a `Socket` object that represents the connection that was made. You can use this object to get input and output streams, as is shown in the following code:

```
InputStream inStream = incoming.getInputStream();
OutputStream outStream = incoming.getOutputStream();
```

Everything that the server sends to the server output stream becomes the input of the client program, and all the output from the client program ends up in the server input stream.

In all the examples in this chapter, we transmit text through sockets. We therefore turn the streams into scanners and writers.

```
var in = new Scanner(inStream);
var out = new PrintWriter(new OutputStreamWriter(outStream), true /* autoFlush */);
```

Let's send the client a greeting:

```
out.println("Hello! Enter BYE to exit.");
```

When you use telnet to connect to this server program at port 8189, you will see this greeting on the terminal screen.

In this simple server, we just read the client's input, a line at a time, and echo it. This demonstrates that the program receives the input. An actual server would obviously compute and return an answer depending on the input.

```
String line = in.nextLine();
out.println("Echo: " + line);
if (line.strip().equals("BYE")) done = true;
```

In the end, we close the incoming socket.

```
incoming.close();
```

That is all there is to it. Every server program, such as an HTTP web server, continues performing this loop:

1. It receives a command from the client ("get me this information") through an incoming data stream.
2. It decodes the client command.
3. It gathers the information that the client requested.
4. It sends the information to the client through the outgoing data stream.

Listing 4.3 is the complete program.

Listing 4.3 v2ch04/server/EchoServer.java

```
1  package v2ch04.server;
2
3  import module java.base;
4
5  /**
6   * This program implements a simple server that listens to port 8189 or a port given as
7   * argument, and echoes back all client input.
8   */
9  class EchoServer {
10     void main(String[] args) throws Exception {
11         int port = args.length >= 1 ? Integer.parseInt(args[0]) : 8189;
12
13         try (var s = new ServerSocket(port); // establish server socket
14                 Socket incoming = s.accept()) { // wait for client connection
15             serve(incoming);
16         }
17     }
18
19     void serve(Socket incoming) throws IOException {
20         try (var in = new Scanner(incoming.getInputStream());
21                 var out = new PrintWriter(incoming.getOutputStream(), true /* autoFlush */)) {
22             out.println("Hello! Enter BYE to exit.");
23
24             // echo client input
25             boolean done = false;
26             while (!done && in.hasNextLine()) {
27                 String line = in.nextLine();
28                 out.println("Echo: " + line);
29                 if (line.strip().equals("BYE")) done = true;
30             }
31         }
32     }
33 }
```

To try it out, compile and run the program. Then use telnet to connect to the server `localhost` (or IP address 127.0.0.1) and port 8189.

If you are connected directly to the Internet, anyone in the world can access your echo server, provided they know your IP address and the magic port number.

When you connect to the port, you will see the message shown in **Figure 4.4**:

```
Hello! Enter BYE to exit.
```

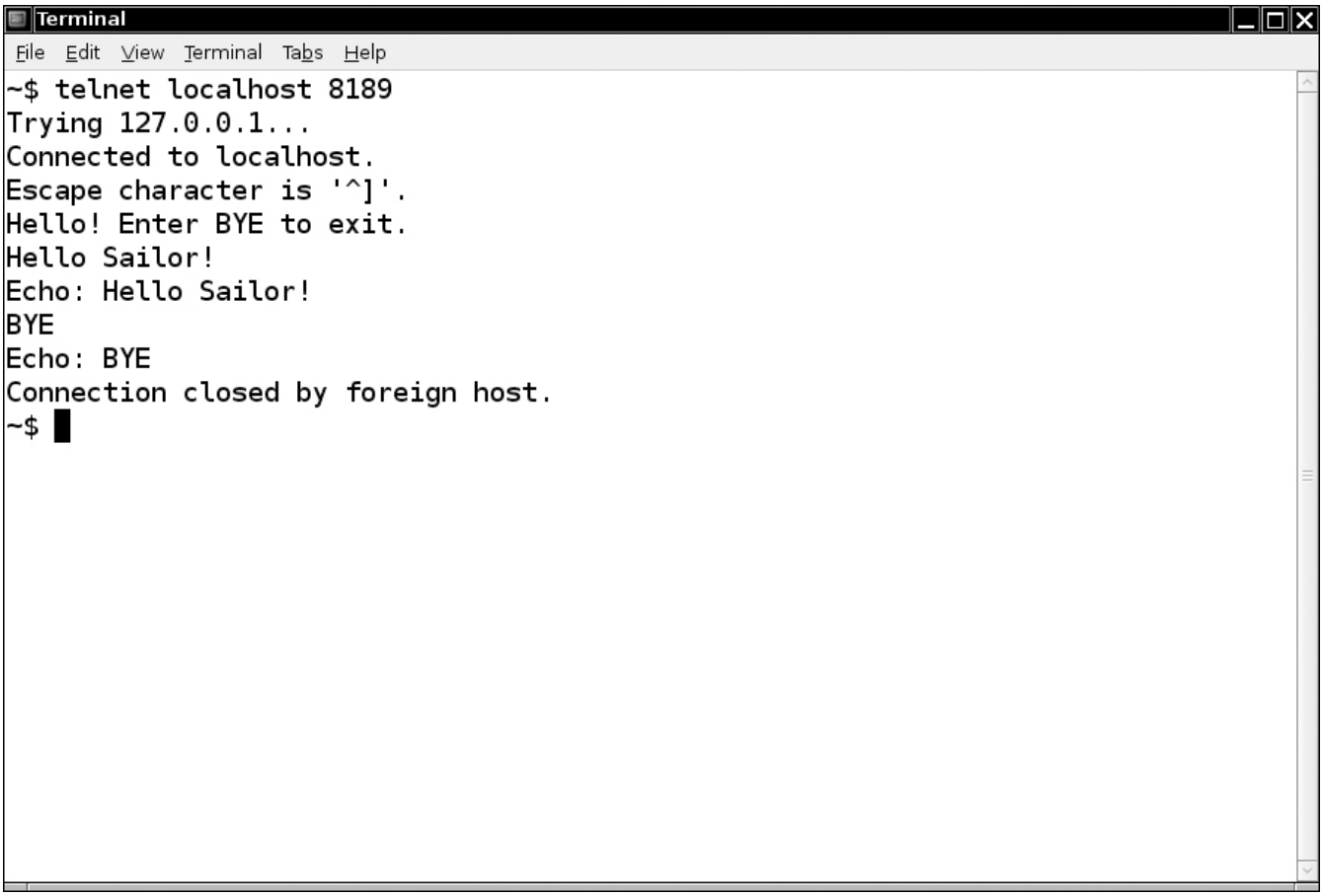


Figure 4.4: Accessing an echo server

Type anything and watch the input echo on your screen. Type `BYE` (all uppercase letters) to disconnect. The server program will terminate as well.

`java.net.ServerSocket` 1.0

- `ServerSocket(int port)`
  creates a server socket that monitors a port.
- `Socket accept()`
  waits for a connection. This method blocks (i.e., idles) the current thread until the connection is made. The method returns a `Socket` object through which the program can communicate with the connecting client.
- `void close()`
  closes the server socket.

4.2.2. Serving Multiple Clients

There is one problem with the simple server in the preceding example. Suppose we want to allow multiple clients to connect to our server at the same time. Typically, a server runs constantly on a server computer, and clients from all over the Internet might want to use it at the same time. Without support for multiple connections, any one client can monopolize the service by connecting to it for a long time. We can do much better through the magic of threads.

Every time we know the program has established a new socket connection—that is, every time the call to `accept()` returns a socket—we will launch a new thread to take care of the connection between the server and *that* client. The main program will just go back and wait for the next connection. It keeps going forever, or until the server process is killed:

```
ExecutorService service = Executors.newVirtualThreadPerTaskExecutor();
while (true) {
    Socket incoming = s.accept();
    service.submit(() -> serve(incoming));
}
```

The `serve` method contains the communication loop with the client:

```
public void serve(Socket incoming) {
    try (var in = new Scanner(incoming.getInputStream());
            var out = new PrintWriter(incoming.getOutputStream(),
            true /* autoFlush */)) {
        Process input and send response
    }
    catch (IOException e) {
        Handle exception
    }
}
```

When each connection starts a new thread, multiple clients can connect to the server at the same time. You can easily check this out.

1. Compile and run the server program (**Listing 4.4**).
2. Open several telnet windows as we have in **Figure 4.5**.
3. Switch between windows and type commands. Note that you can communicate through all of them simultaneously.
4. When done, switch to the window from which you launched the server program and press Ctrl+C to kill it.

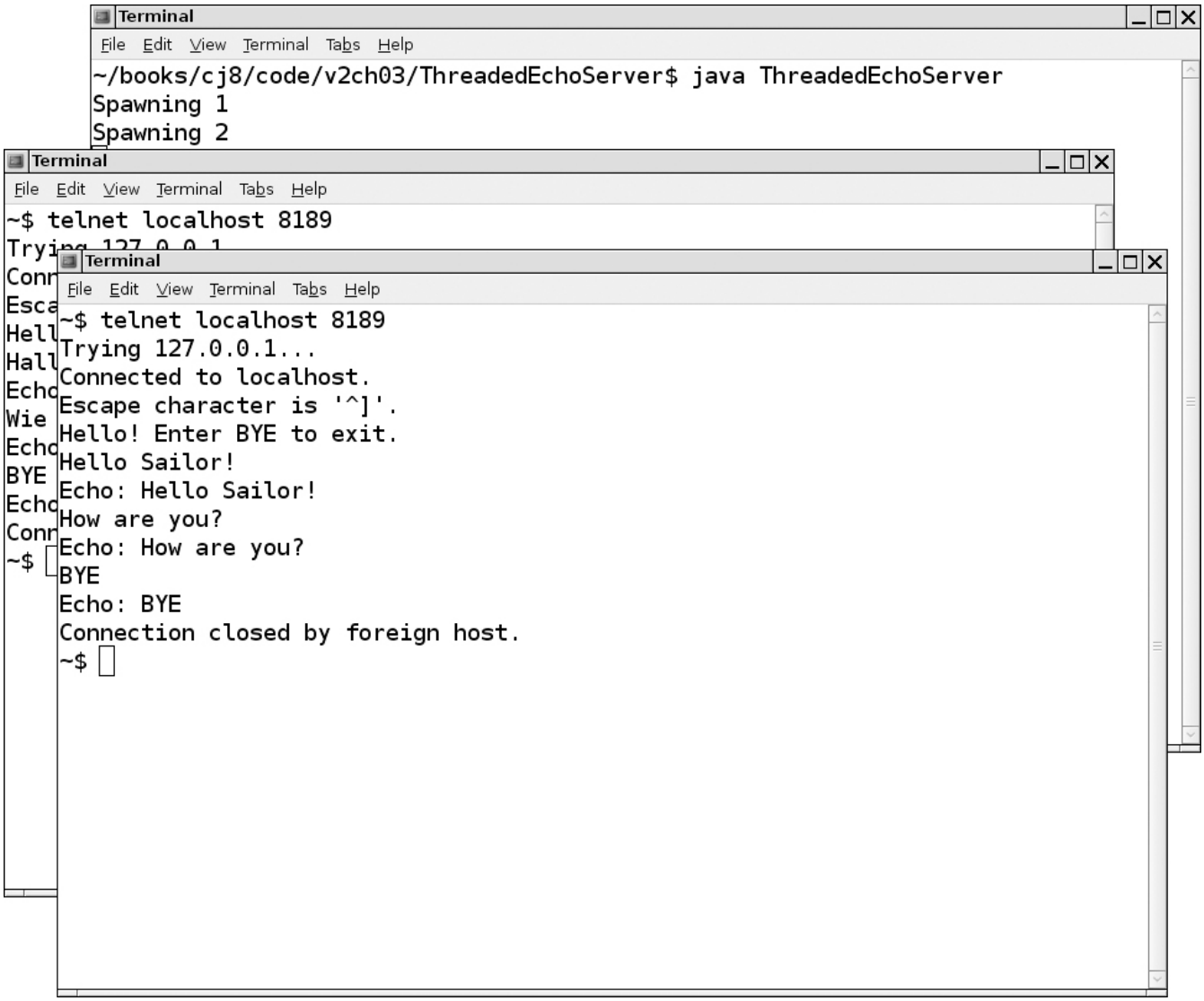


Figure 4.5: Several telnet windows communicating simultaneously

Listing 4.4 v2ch04/threaded/EchoServer.java

```
1  package v2ch04.threaded;
2
3  import module java.base;
4
5  /**
6   * This program implements a multithreaded server that listens to port 8189 and echoes back all
7   * client input.
8   */
9  class EchoServer {
10     void main() throws Exception {
11         try (var s = new ServerSocket(8189)) {
12             int i = 1;
13             ExecutorService service = Executors.newVirtualThreadPerTaskExecutor();
14
15             while (true) {
16                 Socket incoming = s.accept();
17                 IO.println("Spawning " + i);
18                 service.submit(() -> serve(incoming));
19                 i++;
20             }
21         }
22     }
23
```

```
    void serve(Socket incoming) {
        try (var in = new Scanner(incoming.getInputStream());
                var out = new PrintWriter(incoming.getOutputStream(), true /* autoFlush */)) {
            out.println("Hello! Enter BYE to exit.");

            // echo client input
            boolean done = false;
            while (!done && in.hasNextLine()) {
                String line = in.nextLine();
                out.println("Echo: " + line);
                if (line.strip().equals("BYE")) done = true;
            }
        }
        catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

4.2.3. Half-Close

The *half-close* allows one end of a socket connection to terminate its output while still receiving data from the other end.

Here is a typical situation. Suppose you transmit data to the server but you don't know at the outset how much data you have. With a file, you'd just close the file at the end of the data. However, if you close a socket, you immediately disconnect from the server and cannot read the response.

The half-close solves this problem. You can close the output stream of a socket, thereby indicating to the server the end of the requested data, but keep the input stream open.

The client side looks like this:

```
try (var socket = new Socket(host, port)) {
    var in = new Scanner(socket.getInputStream());
    var writer = new PrintWriter(socket.getOutputStream());
    // send request data
    writer.print(. . .);
    writer.flush();
    socket.shutdownOutput();
    // now socket is half-closed
    // read response data
    while (in.hasNextLine()) {
        String line = in.nextLine();
        . . .
    }
}
```

The server side simply reads input until the end of the input stream is reached. Then it sends the response.

Of course, this protocol is only useful for one-shot services such as HTTP where the client connects, issues a request, catches the response, and then disconnects.

`java.net.Socket` 1.0

- `void shutdownInput()` 1.3
- `void shutdownOutput()` 1.3

  set the input or output stream to “end of stream.”
- `boolean isInputShutdown()` 1.4
- `boolean isOutputShutdown()` 1.4

  return `true` if input or output has been shut down.

4.2.4. Interruptible Sockets

When you connect to a socket, the current thread blocks until the connection has been established or a timeout has elapsed. Similarly, when you read data through a socket, the current thread blocks until the operation is successful or has timed out. (There is no timeout for writing data.)

In interactive applications, you would like to give users an option to simply cancel a socket connection that does not appear to produce results. However, if a platform thread blocks on an unresponsive socket, you cannot unblock it by calling `interrupt`.



Note: When using virtual threads, socket operations are interruptible, and you do not need the technique discussed in this section.

To interrupt a socket operation, use a `SocketChannel`, a feature of the `java.nio` package. Open the `SocketChannel` like this:

```
SocketChannel channel = SocketChannel.open(new InetSocketAddress(host, port));
```

A channel does not have associated streams. Instead, it has `read` and `write` methods that make use of `Buffer` objects. (See **Chapter 2** for more information about NIO buffers.) These methods are declared in the interfaces `ReadableByteChannel` and `WritableByteChannel`.

If you don’t want to deal with buffers, you can use the `Scanner` class to read from a `SocketChannel` because `Scanner` has a constructor with a `ReadableByteChannel` parameter:

```
var in = new Scanner(channel);
```

To turn a channel into an output stream, use the static `Channels.newOutputStream` method.

```
OutputStream outStream = Channels.newOutputStream(channel);
```

That’s all you need to do. Whenever a thread is interrupted during an open, read, or write operation, the operation does not block, but is terminated with an exception.

The program in **Listing 4.5** contrasts interruptible and blocking sockets. A server sends numbers and pretends to be stuck after the tenth number. You are asked whether you want to use a socket or a channel. You can also choose between platform and virtual threads. A client thread is started that connects to the server and prints the output. Hit Enter to interrupt the client thread. If you do so within the first ten numbers, you can interrupt the thread.

However, if you use a socket and a platform thread, interruption after the first ten numbers does not work. The blocking thread keeps blocking until the server finally closes the connection. The remedy is to use a channel or virtual threads.

Listing 4.5 v2ch04/interruptible/InterruptibleSocketDemo.java

```
package v2ch04.interruptible;

import module java.base;

/**
 * This program shows how to interrupt a socket channel.
 */
class InterruptibleSocketDemo {
    void main() {
        Thread.ofPlatform().start(new TestServer());
        boolean socket = IO.readln("Socket or channel? (S/C) ").equalsIgnoreCase("S");
        boolean platform = IO.readln("Platform or virtual? (P/V) ").equalsIgnoreCase("P");
        Runnable client = socket ? this::useSocket : this::useChannel;
        Thread clientThread = platform ? Thread.ofPlatform().start(client)
                : Thread.ofVirtual().start(client);

        IO.readln("Hit Enter to interrupt client");
        clientThread.interrupt();
    }

    /**
     * Connects to the test server, using a channel.
     */
    void useChannel() {
        IO.println("Channel:\n");
        try (SocketChannel channel = SocketChannel
                .open(new InetSocketAddress("localhost", 8189));
                Scanner in = new Scanner(channel)) {
            while (!Thread.currentThread().isInterrupted()) {
                IO.print("Reading ");
                if (in.hasNextLine()) {
                    String line = in.nextLine();
                    IO.println(line);
                }
            }
        }
        catch (IOException e) {
            e.printStackTrace();
        }
        finally {
            IO.println("Channel closed\n");
        }
    }

    /**
     * Connects to the test server, using a socket.
     */
    void useSocket() {
        IO.println("Socket:\n");
        try (var sock = new Socket("localhost", 8189);
                Scanner in = new Scanner(sock.getInputStream())) {
```

```
52              while (!Thread.currentThread().isInterrupted()) {
53                  IO.print("Reading ");
54                  if (in.hasNextLine()) {
55                      String line = in.nextLine();
56                      IO.println(line);
57                  }
58              }
59          }
60          catch (IOException e) {
61              e.printStackTrace();
62          }
63          finally {
64              IO.println("Socket closed\n");
65          }
66      }
67
68      /**
69       * A server that listens to port 8189 and sends numbers to the client, simulating a hanging
70       * server after 10 numbers.
71       */
72      static class TestServer implements Runnable {
73          public void run() {
74              try (var s = new ServerSocket(8189); Socket incoming = s.accept()) {
75                  serve(incoming);
76              }
77              catch (Exception e) {
78                  e.printStackTrace();
79                  ;
80              }
81              finally {
82                  IO.println("Closing connection\n");
83              }
84          }
85
86          private void serve(Socket incoming) throws IOException, InterruptedException {
87              int counter = 0;
88              try (var out = new PrintWriter(incoming.getOutputStream(), true /* autoFlush */)) {
89                  while (counter < 100) {
90                      counter++;
91                      if (counter <= 10) out.println(counter);
92                      Thread.sleep(100);
93                  }
94              }
95          }
96      }
97  }
```

`java.net.InetSocketAddress` 1.4

- `InetSocketAddress(String hostname, int port)`
  constructs an address object with the given host and port, resolving the host name during construction. If the host name cannot be resolved, the address object's `unresolved` property is set to `true`.
- `boolean isUnresolved()`
  returns `true` if this address object could not be resolved.

`java.nio.channels.SocketChannel` 1.4

- `static SocketChannel open(SocketAddress address)`
  opens a socket channel and connects it to a remote address.

`java.nio.channels.Channels` 1.4

- `static InputStream newInputStream(ReadableByteChannel channel)`
  constructs an input stream that reads from the given channel.
- `static OutputStream newOutputStream(WritableByteChannel channel)`
  constructs an output stream that writes to the given channel.

4.2.5. Secure Socket Communication

When you implement clients and servers across the Internet and you exchange confidential information, you do not want to use plain text traffic. Instead, you should use a secure communication mechanism. There are three aspects to secure communication:

- The communication is encrypted, so it cannot be read by third parties.
- The identity of the server is authenticated with a digital signature that the client verifies. Optionally, the client can also provide a digital signature, but this is not common for web applications.
- Transmitted data are digitally signed by the sender and verified by the receiver, guaranteeing that no third party modified or substituted the data.

The TLS (Transport Layer Security) protocol is the most common mechanism for secure communication. It is a successor to the SSL (Secure Socket Layer) protocol.

In Java, you use classes from the `javax.net.ssl` package to establish secure connections. Here is how to construct a socket for a secure connection:

```
SocketFactory factory = SSLSocketFactory.getDefault();
Socket s = factory.createSocket(host, port);
```

For secure server sockets, you also use a factory:

```
ServerSocketFactory factory = SSLServerSocketFactory.getDefault();
ServerSocket s = factory.createServerSocket(port);
```

If you need to configure the secure connection parameters of the client or server socket, cast the object returned by the factory to a `SSLSocket` or `SSLServerSocket`:

```
((SSLServerSocket) s).setNeedClientAuth(true);
```

The client and server encrypt the traffic, using a shared encryption key. However, they need to know that they are really talking to each other and not to a “man in the middle.” The server presents a certificate to vouch for its authenticity. The certificate needs to be signed by a certificate authority known to the client. The JDK has a built-in list of known certificate authorities. You would normally obtain a server certificate from one of them and install it in the server. However, for testing, we will generate our own certificates.

See **Chapter 9** for a detailed description of certificates and the `keytool` program used to manage them. Here are the commands to generate the certificates and place them in a "key store" for the server and client.

First, generate the server certificate:

```
keytool -genkeypair -alias servercert -keyalg RSA -keysize 2048 -sigalg SHA256withRSA \
    -keystore server.jks -storepass secret -ext san=ip:127.0.0.1,dns:localhost
```

You will be asked a number of questions about the organization in charge of the server. For this demonstration, you can leave them all to the default value of "unknown".

Next, export the certificate and import it into the client store.

```
keytool -exportcert -keystore server.jks -storepass secret -alias servercert -rfc \
    -file servercert.pem
keytool -import -trustcacerts -file servercert.pem -keystore client.jks \
    -storepass secret -keypass secret
```

When importing the certificate, which is not signed by one of the known certificate authorities, you will be asked whether you trust it. Answer with `yes`.

Now start the server. You need to specify the keystore containing the server certificate.

```
java -Djavax.net.ssl.keyStore=server.jks -Djavax.net.ssl.keyStorePassword=secret \
    v2ch04.ssl.EchoServer
```

Then run the client, specifying the keystore containing the certificate that the client trusts. This would not be necessary if the certificate was signed by a certificate authority that is known to the JDK.

```
java -Djavax.net.ssl.trustStore=client.jks -Djavax.net.ssl.trustStorePassword=secret \
    v2ch04.ssl.EchoClient
```



Tip: If there are connection problems, try running the client with the `-Djavax.net.debug=ssl` command-line option.

Listing 4.6 v2ch04/ssl/EchoClient.java

```
1  package v2ch04.ssl;
2
3  import module java.base;
4
5  /**
6   * This program connects to a server via SSL and echoes the server responses.
7   */
8  class EchoClient {
9      void main(String[] args) throws Exception {
10         int port = args.length >= 1 ? Integer.parseInt(args[0]) : 8189;
11         SocketFactory factory = SSLSocketFactory.getDefault();
12         String message = """
13 Hello
```

```
World
BYE
""";
        try (Socket s = factory.createSocket(InetAddress.getLocalHost(), port);
                OutputStream os = s.getOutputStream();
                var in = new Scanner(s.getInputStream())) {
            os.write(message.getBytes());
            os.flush();
            boolean done = false;
            while (!done) {
                String line = in.nextLine();
                System.out.println(line);
                if (line.equals("Echo: BYE")) done = true;
            }
        }
    }
}
```

Listing 4.7 v2ch04/ssl/EchoServer.java

```
package v2ch04.ssl;

import module java.base;

/**
 * This program implements a simple server that listens to port 8189
 * or a port given as argument, and echoes back all client input.
 */
class EchoServer {
    void main(String[] args) throws Exception {
        int port = args.length >= 1 ? Integer.parseInt(args[0]) : 8189;
        // establish server socket
        ServerSocketFactory factory = SSLServerSocketFactory.getDefault();
        try (ServerSocket s = factory.createServerSocket(port)) {
            ExecutorService service = Executors.newVirtualThreadPerTaskExecutor();

            while (true) {
                // wait for client connection
                Socket incoming = s.accept();
                service.submit(() -> serve(incoming));
            }
        }
    }

    void serve(Socket incoming) {
        try (var in = new Scanner(incoming.getInputStream());
             var out = new PrintWriter(incoming.getOutputStream(),
                     true /* autoFlush */)) {
            out.println("Hello! Enter BYE to exit.");

            // echo client input
            boolean done = false;
            while (!done && in.hasNextLine()) {
                String line = in.nextLine();
                out.println("Echo: " + line);
                if (line.strip().equals("BYE"))
                    done = true;
```

```
38              }
39          }
40          catch (IOException e) {
41              e.printStackTrace();
42          }
43      }
44  }
```

`javax.net.ServerSocketFactory` 1.4

- `static ServerSocket createServerSocket(int port)`
  yields a server socket for the given port.

`javax.net.ssl.SSLServerSocketFactory` 1.4

- `static ServerSocketFactory getDefault()`
  returns a factory that produces `SSLServerSocket` instances.

`javax.net.ssl.SSLSocketFactory` 1.4

- `static SocketFactory getDefault()`
  returns a factory that produces `SSLSocket` instances.

`javax.net.SocketFactory` 1.4

- `static Socket createSocket(InetAddress host, int port)`
- `static Socket createSocket(String host, int port)`
  yield a socket for connecting to the given host and port.

4.3. Getting Web Data

To access web servers in a Java program, you will want to work at a higher level than socket connections and HTTP requests. In the following sections, we discuss the classes that the Java library provides for this purpose.

4.3.1. URLs and URIs

The `java.net` package makes a useful distinction between URLs (uniform resource *locators*) and URIs (uniform resource *identifiers*).

A URI is a purely syntactical construct that contains the various parts of the string specifying a web resource. A URL is a special kind of URI, namely one with sufficient information to *locate* a resource. Other URIs, such as

```
mailto:cay@horstmann.com
```

are not locators—there is no data to locate from this identifier. Such a URI is called a URN (uniform resource *name*).

In the Java library, the `URI` class has no methods for accessing the resource that the identifier specifies—its sole purpose is parsing. In contrast, the `URL` class can open a stream to the resource. For that reason, the `URL` class

only works with schemes that the Java library knows how to handle, such as `http:`, `https:`, `ftp:`, the local file system (`file:`), and JAR files (`jar:`).

You can construct a `URL` object from a `URI`:

```
var url = new URI(urlString).toURL();
```



Caution: The `URL` class has constructors that were deprecated in Java 20. They have various technical issues that make it possible to construct invalid URLs. Instead, first construct a URI, and then obtain the URL, as shown above.

If you simply want to fetch the contents of the resource, use the `openStream` method of the `URL` class. This method yields an `InputStream` object. Use it in the usual way—for example, to construct a `Scanner`:

```
InputStream inStream = url.openStream();
var in = new Scanner(inStream);
```

In the next section, you will see how to use the `URLConnection` class for more control over the connection.

Let us return to URIs. To see why parsing is not trivial, consider how complex URIs can be. For example,

```
https://google.com?q=Beach+Chalet
ftp://username:password@ftp.yourserver.com/pub/file.txt
```

The URI specification gives the rules for the makeup of these identifiers. A URI has the syntax

```
[scheme:]schemeSpecificPart[#fragment]
```

Here, the brackets [ ] enclose optional parts, and the `:` and `#` are included literally in the identifier.

If the `scheme:` part is present, the URI is called *absolute*. Otherwise, it is called *relative*.

An absolute URI is *opaque* if the `schemeSpecificPart` does not begin with a `/` such as

```
mailto:cay@horstmann.com
```

All absolute nonopaque URIs and all relative URIs are *hierarchical*. Examples are

```
http://horstmann.com/index.html
../../java/net/Socket.html#Socket()
```

The `schemeSpecificPart` of a hierarchical URI has the structure

```
[//authority][path][?query]
```

where, again, the brackets [ ] enclose optional parts.

For server-based URIs, the authority part has the form

```
[user-info@]host[:port]
```

The `port` must be an integer.

RFC 2396, which standardizes URIs, also supports a registry-based mechanism in which the authority has a different format, but this is not in common use.

One of the purposes of the `URI` class is to parse an identifier and break it up into its components. You can retrieve them with the methods

```
getScheme
getSchemeSpecificPart
getAuthority
getUserInfo
getHost
getPort
getPath
getQuery
getFragment
```

The other purpose of the `URI` class is the handling of absolute and relative identifiers. If you have an absolute URI such as

```
https://docs.mycompany.com/api/java/net/ServerSocket.html
```

and a relative URI such as

```
../../java/net/Socket.html#Socket()
```

then you can combine the two into an absolute URI.

```
https://docs.mycompany.com/api/java/net/Socket.html#Socket()
```

This process is called *resolving* a relative URL.

The opposite process is called *relativization*. For example, suppose you have a *base* URI

```
https://docs.mycompany.com/api
```

and a URI

```
https://docs.mycompany.com/api/java/lang/String.html
```

Then the relativized URI is

```
java/lang/String.html
```

The URI class supports both of these operations:

```
relative = base.relativize(combined);
combined = base.resolve(relative);
```

4.3.2. Using a `URLConnection` to Retrieve Information

If you want additional information about a web resource, you should use the `URLConnection` class, which gives you much more control than the basic `URL` class.

When working with a `URLConnection` object, you must carefully schedule your steps.

1. Call the `openConnection` method of the `URL` class to obtain the `URLConnection` object:

   ```
   URLConnection connection = url.openConnection();
   ```

2. Set any request properties, using the methods

   ```
   setDoInput
   setDoOutput
   setIfModifiedSince
   setUseCaches
   setAllowUserInteraction
   setRequestProperty
   setConnectTimeout
   setReadTimeout
   ```

   We discuss these methods later in this section and in the API notes.
3. Connect to the remote resource by calling the `connect` method:

   ```
   connection.connect();
   ```

   Besides making a socket connection to the server, this method also queries the server for *header information.*
4. After connecting to the server, you can query the header information. Two methods, `getHeaderFieldKey` and `getHeaderField`, enumerate all fields of the header. The method `getHeaderFields` gets a standard `Map` object containing the header fields. For your convenience, the following methods query standard fields:

   ```
   getContentType
   getContentLength
   getContentEncoding
   getDate
   getExpiration
   getLastModified
   ```

5. Finally, you can access the resource data. Use the `getInputStream` method to obtain an input stream for reading the information. (This is the same input stream that the `openStream` method of the `URL` class re-

turns.) The other method, `getContent`, isn't very useful in practice. The objects that are returned by standard content types such as `text/plain` and `image/gif` require classes in the `com.sun` hierarchy for processing. You could register your own content handlers, but we do not discuss this technique in our book.



Caution: Some programmers form a wrong mental image when using the `URLConnection` class, thinking that the `getInputStream` and `getOutputStream` methods are similar to those of the `Socket` class. But that isn't quite true. The `URLConnection` class does quite a bit of magic behind the scenes—in particular, the handling of request and response headers. For that reason, it is important that you follow the setup steps for the connection.

Let us now look at some of the `URLConnection` methods in detail. Several methods set properties of the connection before connecting to the server. The most important ones are `setDoInput` and `setDoOutput`. By default, the connection yields an input stream for reading from the server but no output stream for writing. If you want an output stream (for example, for posting data to a web server), you need to call

```
connection.setDoOutput(true);
```

Next, you may want to set some of the request headers. The request headers are sent together with the request command to the server. Here is an example:

```
GET www.server.com/index.html HTTP/1.0
Referer: https://www.somewhere.com/links.html
Proxy-Connection: Keep-Alive
User-Agent: Mozilla/5.0 (X11; U; Linux i686; en-US; rv:1.8.1.4)
Host: www.server.com
Accept: text/html, image/gif, image/jpeg, image/png, */*
Accept-Language: en
Accept-Charset: iso-8859-1,*,utf-8
Cookie: orangemilano=192218887821987
```

The `setIfModifiedSince` method tells the connection that you are only interested in data modified since a certain date.

Finally, you can use the catch-all `setRequestProperty` method to set any name/value pair that is meaningful for the particular protocol. For the format of the HTTP request headers, see RFC 2616. Some of these properties are not well documented and are passed around by word of mouth from one programmer to the next. For example, if you want to access a password-protected web page, you must do the following:

1. Concatenate the username, a colon, and the password.

   ```
   String input = username + ":" + password;
   ```

2. Compute the Base64 encoding of the resulting string. (The Base64 encoding encodes a sequence of bytes into a sequence of printable ASCII characters.)

   ```
   Base64.Encoder encoder = Base64.getEncoder();
   String encoding = encoder.encodeToString(input.getBytes());
   ```

3. Call the `setRequestProperty` method with a name of `"Authorization"` and the value `"Basic " + encoding`.

```
connection.setRequestProperty("Authorization", "Basic " + encoding);
```



Tip: You just saw how to access a password-protected web page. To access a password-protected file by FTP, use an entirely different method: Construct a URL of the form

```
ftp://username:password@ftp.yourserver.com/pub/file.txt
```

Once you call the `connect` method, you can query the response header information. First, let's see how to enumerate all response header fields. The implementors of this class felt a need to express their individuality by introducing yet another iteration protocol. The call

```
String key = connection.getHeaderFieldKey(n);
```

gets the `n`th key from the response header, where `n` starts from `1`! It returns `null` if `n` is less than 1 or greater than the total number of header fields. There is no method to return the number of fields; you simply keep calling `getHeaderFieldKey` until you get `null`. Similarly, the call

```
String value = connection.getHeaderField(n);
```

returns the `n`th value.

The method `getHeaderFields` returns a `Map` of response header fields.

```
Map<String,List<String>> headerFields = connection.getHeaderFields();
```

Here is a set of response header fields from a typical HTTP request:

```
Date: Wed, 27 Aug 2008 00:15:48 GMT
Server: Apache/2.2.2 (Unix)
Last-Modified: Sun, 22 Jun 2008 20:53:38 GMT
Accept-Ranges: bytes
Content-Length: 4813
Connection: close
Content-Type: text/html
```



Note: Even though `connection.getHeaderFieldKey(0)` returns `null`, you can call `connection.getHeaderField(0)` to get the response status line (such as `"HTTP/1.1 200 OK"`). You also get it when you call `headerFields.get(null)`.

As a convenience, six methods query the values of the most common header types and convert them to numeric types when appropriate. **Table 4.1** shows these convenience methods. The methods with return type `long` return the number of seconds since January 1, 1970 GMT.

Table 4.1: Convenience Methods for Response Header Values

Key Name	Method Name	Return Type
`Date`	`getDate`	`long`
`Expires`	`getExpiration`	`long`
`Last-Modified`	`getLastModified`	`long`
`Content-Length`	`getContentLength`	`int`
`Content-Type`	`getContentType`	`String`
`Content-Encoding`	`getContentEncoding`	`String`

The program in **Listing 4.8** lets you experiment with URL connections. Supply a URL and an optional username and password on the command line when running the program, for example:

```
java urlConnection.URLConnectionDemo https://horstmann.com/corejava/secret user password
```

The program prints

- All keys and values of the header
- The return values of the six convenience methods in **Table 4.1**
- The first ten lines of the requested resource

Listing 4.8 v2ch04/urlConnection/URLConnectionDemo.java

```
1  package v2ch04.urlConnection;
2
3  import module java.base;
4
5  /**
6   * This program connects to an URL and displays the response header data and the first 10 lines
7   * of the requested data.
8   *
9   * Supply the URL and an optional username and password (for HTTP basic authentication) on the
10  * command line.
11  */
12 class URLConnectionDemo {
13     void main(String[] args) throws Exception {
14         String urlName = args.length > 0 ? args[0] : "http://horstmann.com";
15
16         var url = new URI(urlName).toURL();
17         URLConnection connection = url.openConnection();
18
19         // set username, password if specified on command line
20
21         if (args.length > 2) {
22             String username = args[1];
23             String password = args[2];
24             String input = username + ":" + password;
25             Base64.Encoder encoder = Base64.getEncoder();
```

```
26              String encoding = encoder.encodeToString(input.getBytes());
27              connection.setRequestProperty("Authorization", "Basic " + encoding);
28          }
29
30          connection.connect();
31
32          // print header fields
33
34          Map<String, List<String>> headers = connection.getHeaderFields();
35          for (Map.Entry<String, List<String>> entry : headers.entrySet()) {
36              String key = entry.getKey();
37              for (String value : entry.getValue())
38                  IO.println(key + ": " + value);
39          }
40
41          // print convenience functions
42
43          IO.println("----------");
44          IO.println("getContentType: " + connection.getContentType());
45          IO.println("getContentLength: " + connection.getContentLength());
46          IO.println("getContentEncoding: " + connection.getContentEncoding());
47          IO.println("getDate: " + connection.getDate());
48          IO.println("getExpiration: " + connection.getExpiration());
49          IO.println("getLastModifed: " + connection.getLastModified());
50          IO.println("----------");
51
52          String encoding = connection.getContentEncoding();
53          if (encoding == null) encoding = "UTF-8";
54          try (var in = new Scanner(connection.getInputStream(), encoding)) {
55              // print first ten lines of contents
56
57              for (int n = 1; in.hasNextLine() && n <= 10; n++)
58                  IO.println(in.nextLine());
59              if (in.hasNextLine()) IO.println(". . .");
60          }
61      }
62  }
```

`java.net.URL` **1.0**

- `InputStream openStream()`
  opens an input stream for reading the resource data.
- `URLConnection openConnection()`
  returns a `URLConnection` object that manages the connection to the resource.

`java.net.URLConnection` **1.0**

- `void setDoInput(boolean doInput)`
- `boolean getDoInput()`
  If `doInput` is `true`, the user can receive input from this `URLConnection`.
- `void setDoOutput(boolean doOutput)`
- `boolean getDoOutput()`
  If `doOutput` is `true`, the user can send output to this `URLConnection`.

- `void setIfModifiedSince(long time)`
- `long getIfModifiedSince()`
  The `ifModifiedSince` property configures this `URLConnection` to fetch only data modified since a given time. The time is given in seconds since midnight, GMT, January 1, 1970.
- `void setConnectTimeout(int timeout)` **5.0**
- `int getConnectTimeout()` **5.0**
  set or get the timeout for the connection (in milliseconds). If the timeout has elapsed before a connection was established, the `connect` method of the associated input stream throws a `SocketTimeoutException`.
- `void setReadTimeout(int timeout)` **5.0**
- `int getReadTimeout()` **5.0**
  set or get the timeout for reading data (in milliseconds). If the timeout has elapsed before a read operation was successful, the `read` method throws a `SocketTimeoutException`.
- `void setRequestProperty(String key, String value)`
  sets a request header field.
- `Map<String,List<String>> getRequestProperties()` **1.4**
  returns a map of request properties. All values for the same key are placed in a list.
- `void connect()`
  connects to the remote resource and retrieves the response header information.
- `Map<String,List<String>> getHeaderFields()` **1.4**
  returns a map of response headers. All values for the same key are placed in a list.
- `String getHeaderFieldKey(int n)`
  gets the key for the `n` th response header field, or `null` if `n` is ≤ `0` or greater than the number of response header fields.
- `String getHeaderField(int n)`
  gets value of the `n` th response header field, or the response status line if `n` is 0 or `null` if `n` is < `0` or greater than the number of response header fields.
- `int getContentLength()`
  gets the content length if available, or `-1` if unknown.
- `String getContentType()`
  gets the content type, such as `text/plain` or `image/gif`.
- `String getContentEncoding()`
  gets the content encoding, such as `gzip`. This value is not commonly used, because the default `identity` encoding is not supposed to be specified with a `Content-Encoding` header.
- `long getDate()`
- `long getExpiration()`
- `long getLastModifed()`
  get the date of creation, expiration, and last modification of the resource. The dates are specified as seconds since midnight, GMT, January 1, 1970.
- `InputStream getInputStream()`
- `OutputStream getOutputStream()`
  return a stream for reading from the resource or writing to the resource.
- `Object getContent()`
  selects the appropriate content handler to read the resource data and convert it into an object. This method is not useful for reading standard types such as `text/plain` or `image/gif` unless you install your own content handler.

4.3.3. Posting Form Data

In the preceding section, you saw how to read data from a web server. Now we will show you how your programs can send data back to a server.

To send information from a web browser to a server, a user fills out a *form*, like the one in **Figure 4.6**.

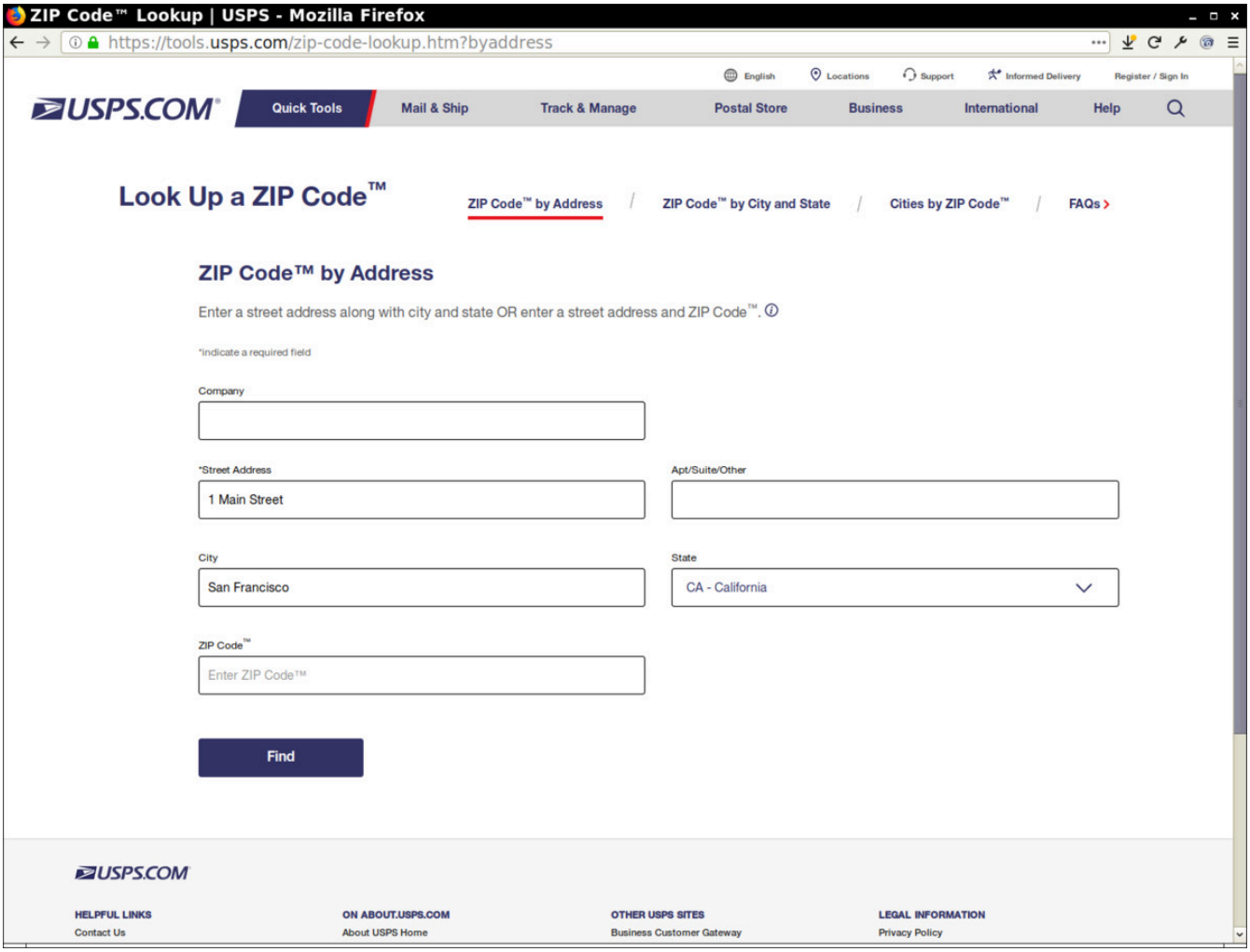


Figure 4.6: An HTML form

When the user clicks the Submit button, the text in the text fields and the settings of any checkboxes, radio buttons, and other input elements are sent back to the server. The server processes the submitted values.

The server then produces another HTML page and sends it back to the browser. This sequence is illustrated in **Figure 4.7**. The response page can contain new information (for example, in an information-search program) or just an acknowledgment. The web browser then displays the response page.

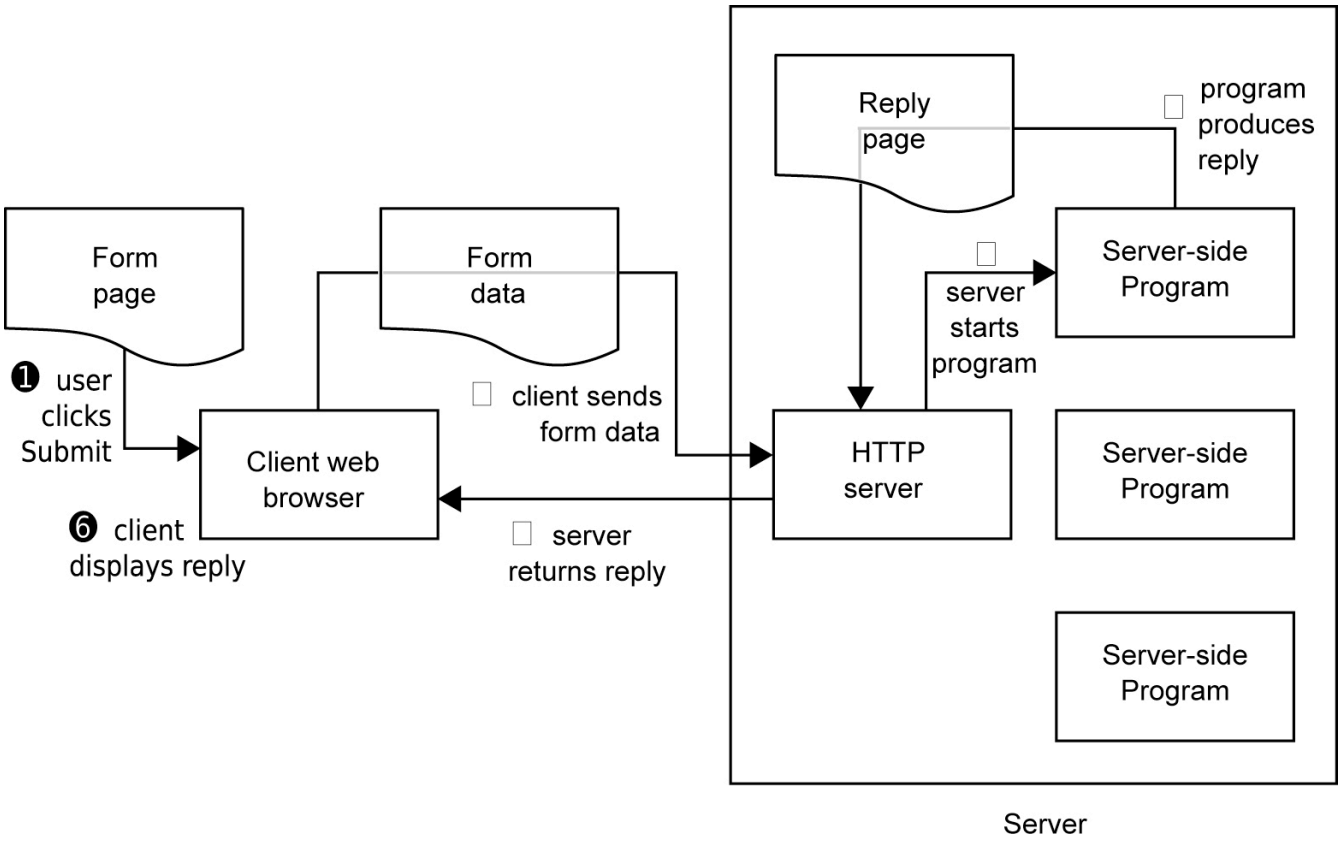


Figure 4.7: Data flow during execution of a form submission

We do not discuss the implementation of server-side programs in this book. Our interest is merely in writing client programs that interact with existing server-side programs.

Two commands, called `GET` and `POST`, are commonly used to send information to a server.

In the `GET` command, you simply attach query parameters to the end of the URL. The URL has the form

```
https://host/path?query
```

Each parameter has the form `name=value`. Parameters are separated by `&` characters. Parameter values are encoded using the *URL encoding* scheme, following these rules:

- Leave the characters `A` through `Z`, `a` through `z`, `0` through `9`, and `. - ~ _` unchanged.
- Replace all spaces with `+` characters.
- Encode all other characters into UTF-8 and encode each byte by a `%`, followed by a two-digit hexadecimal number.

For example, to transmit *San Francisco, CA*, you use `San+Francisco%2c+CA`, as the hexadecimal number `2c` is the UTF-8 code of the `','` character.

This encoding keeps any intermediate programs from messing with spaces and other special characters.

At the time of this writing, the Google Maps site (**https://www.google.com/maps**) accepts query parameters with names `q` and `hl` whose values are the location query and the human language of the response. To get a map of 1 Market Street in San Francisco, with a response in German, use the following URL:

```
https://www.google.com/maps?q=1+Market+Street+San+Francisco&hl=de
```

Very long query strings can look unattractive in browsers, and older browsers and proxies have a limit on the number of characters that you can include in a `GET` request. For that reason, a `POST` request is often used for forms with a lot of data. In a `POST` request, you do not attach parameters to a URL; instead, you get an output stream from the `URLConnection` and write name/value pairs to the output stream. You still have to URL-encode the values and separate them with `&` characters.

Let us look at this process in detail. To post data to a server-side program, first establish a `URLConnection`:

```
URL url = new URI("https://host/path").toURL();
URLConnection connection = url.openConnection();
```

Then, call the `setDoOutput` method to set up the connection for output:

```
connection.setDoOutput(true);
```

Next, call `getOutputStream` to get a stream through which you can send data to the server. If you are sending text to the server, it is convenient to wrap that stream into a `PrintWriter`.

```
var out = new PrintWriter(connection.getOutputStream());
```

Now you are ready to send data to the server:

```
out.print(name1 + "=" + URLEncoder.encode(value1, StandardCharsets.UTF_8) + "&");
out.print(name2 + "=" + URLEncoder.encode(value2, StandardCharsets.UTF_8));
```

Close the output stream:

```
out.close();
```

Finally, call `getInputStream` and read the server response.



Caution: The static methods `URLEncoder.encode` and `URLDecoder.decode` have a version that uses the default charset, but they have been deprecated in Java 6 because they should always be used with UTF-8. Sadly, they are still deprecated even though Java 18 changed the default character encoding to UTF-8.

Let's run through a practical example. The web site at **https://tools.usps.com/zip-code-lookup.htm?byaddress** contains a form to find the zip code for a street address (see **Figure 4.6**). To use this form in a Java program, you need to know the URL and the parameters of the `POST` request.

You could get that information by looking at the HTML code of the form, but it is usually easier to "spy" on a request with a network monitor. Most browsers have a network monitor as part of their development toolkit. For example, **Figure 4.8** shows a screen capture of the Firefox network monitor when submitting data to our example web site. You can find out the submission URL as well as the parameter names and values.

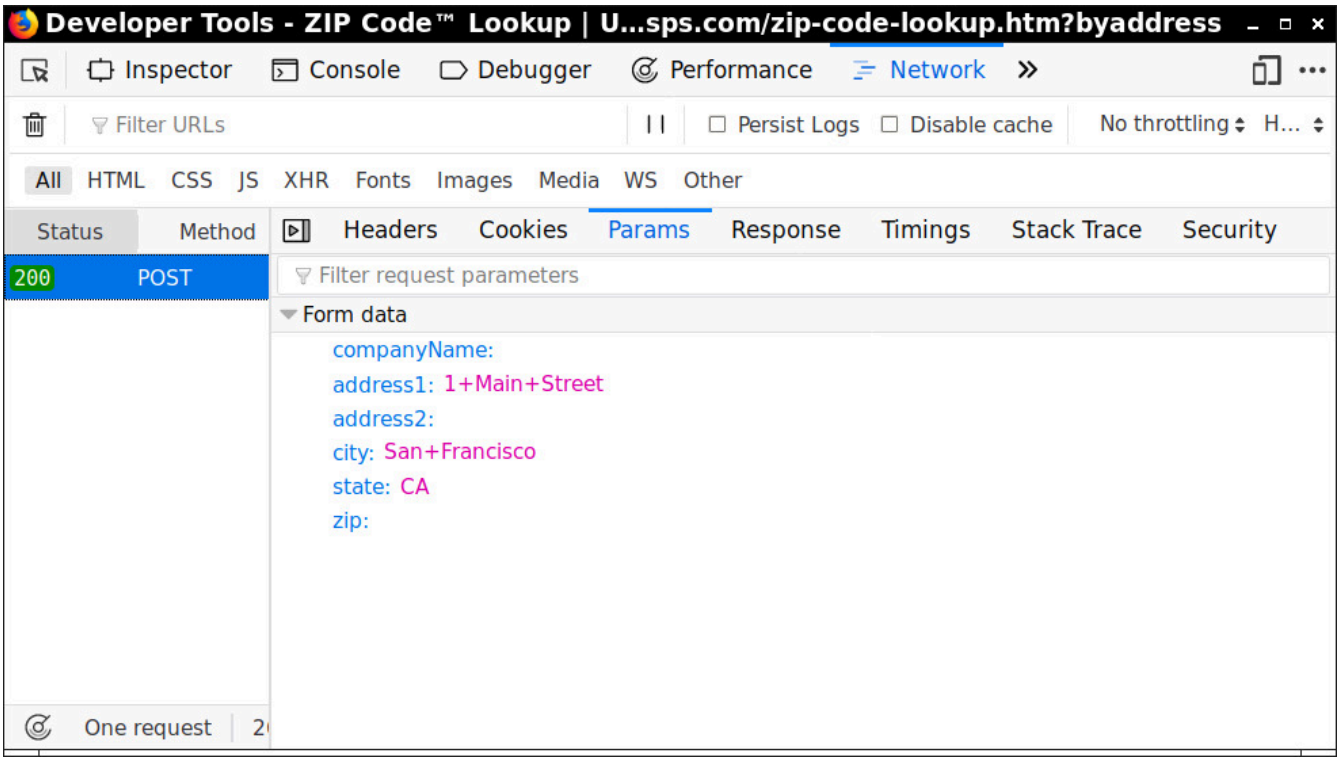


Figure 4.8: Monitoring the submission of a form

When posting form data, the HTTP header includes the content type:

```
Content-Type: application/x-www-form-urlencoded
```

You can also post data in other formats. For example, when sending data in JavaScript Object Notation (JSON), set the content type to `application/json`.

The header for a `POST` must also include the content length, for example

```
Content-Length: 124
```

The program in **Listing 4.9** sends `POST` form data to any server-side program. Place the data into a `.properties` file such as the following:

```
url=https://tools.usps.com/tools/app/ziplookup/zipByAddress
User-Agent=HTTPie/0.9.2
address1=1 Market Street
address2=
city=San Francisco
state=CA
companyName=
. . .
```

The program removes the `url` and `User-Agent` entries and sends all others to the `doPost` method.

In the `doPost` method, we first open the connection and set the user agent. (The zip code service does not work with the default `User-Agent` request parameter which contains the string `Java`, perhaps because the postal service doesn't want to serve programmatic requests.)

Then we call `setDoOutput(true)`, and open the output stream. We then enumerate all keys and values. For each of them, we send the `key`, `=` character, `value`, and `&` separator character:

```
out.print(key);
out.print("=");
out.print(URLEncoder.encode(value, StandardCharsets.UTF_8));
if (more pairs) out.print("&");
```

When switching from writing to reading any part of the response, the actual interaction with the server happens. The `Content-Length` header is set to the size of the output. The `Content-Type` header is set to `application/x-www-form-urlencoded` unless a different content type was specified. The headers and data are sent to the server. Then the response headers and server response are read and can be queried. In our example program, this switch happens in the call to `connection.getContentEncoding()`.

There is one twist with reading the response. If a server-side error occurs, the call to `connection.getInputStream()` throws a `FileNotFoundException`. However, the server still sends an error page back to the browser (such as the ubiquitous "Error 404—page not found"). To capture this error page, call the `getErrorStream` method:

```
InputStream err = connection.getErrorStream();
```



Note: The `getErrorStream` method, as well as several other methods in this program, belong to the `HttpURLConnection` subclass of `URLConnection`. If you make a request to an URL that starts with `http://` or `https://`, you can cast the resulting connection object to `HttpURLConnection`.

When you send `POST` data to a server, it can happen that the server-side program responds with a *redirect*: a different URL that should be called to get the actual information. The server could do that because the information is available elsewhere, or to provide a bookmarkable URL. The `HttpURLConnection` class can handle redirects in most cases.



Note: If cookies need to be sent from one site to another in a redirect, you can configure the global cookie handler like this:

```
CookieHandler.setDefault(new CookieManager(null, CookiePolicy.ACCEPT_ALL));
```

Then cookies will be properly included in the redirect.

Even though redirects are usually automatically handled, there are some situations where you need to do them yourself. Automatic redirects between HTTP and HTTPS are not supported for security reasons. Redirects can also fail for more subtle reasons. For example, an earlier version of the zip code service used a redirect. Recall that we set the `User-Agent` request parameter so that the post office didn't think we made a request via the Java API. While it is possible to set the user agent to a different string in the initial request, that setting is not used in automatic redirects. Instead, automatic redirects always send a generic user agent string that contains the word `Java`.

In such situations, you can manually carry out the redirects. Before connecting the server, turn off automatic redirects:

```
connection.setInstanceFollowRedirects(false);
```

After making the request, get the response code:

```
int responseCode = connection.getResponseCode();
```

Check if it is one of

```
HttpURLConnection.HTTP_MOVED_PERM
HttpURLConnection.HTTP_MOVED_TEMP
HttpURLConnection.HTTP_SEE_OTHER
```

In that case, get the `Location` response header to obtain the URL for the redirect. Then disconnect and make another connection to the new URL:

```
String location = connection.getHeaderField("Location");
if (location != null) {
    URL base = connection.getURL();
    connection.disconnect();
    URL url = base.toURI().resolve(location).toURL();
    connection = (HttpURLConnection) url.openConnection();
    . . .
}
```

The techniques that this program illustrates can be useful whenever you need to query information from an existing web site. Simply find out the parameters that you need to send, and then strip out the HTML tags and other unnecessary information from the reply.

Listing 4.9 v2ch04/post/PostDemo.java

```
1  package v2ch04.post;
2
3  import module java.base;
4
5  /**
6   * This program demonstrates how to use the URLConnection class for a POST request.
7   */
8  public class PostDemo {
9      void main(String[] args) throws Exception {
10         String propsFilename = args.length > 0 ? args[0] : "v2ch04/post/post.properties";
11         var props = new Properties();
12         try (Reader in = Files.newBufferedReader(Path.of(propsFilename))) {
13             props.load(in);
14         }
15         String urlString = props.remove("url").toString();
16         Object userAgent = props.remove("User-Agent");
17         Object redirects = props.remove("redirects");
18         CookieHandler.setDefault(new CookieManager(null, CookiePolicy.ACCEPT_ALL));
19         String result = doPost(new URI(urlString).toURL(), props,
20                 userAgent == null ? null : userAgent.toString(),
21                 redirects == null ? -1 : Integer.parseInt(redirects.toString()));
```

```
        IO.println(result);
    }

    /**
     * Do an HTTP POST.
     * @param url the URL to post to
     * @param nameValuePairs the query parameters
     * @param userAgent the user agent to use, or null for the default user agent
     * @param redirects the number of redirects to follow manually, or -1 for automatic
     *        redirects
     * @return the data returned from the server
     * @throws URISyntaxException
     */
    String doPost(URL url, Map<Object, Object> nameValuePairs, String userAgent, int redirects)
            throws IOException, URISyntaxException {
        var connection = (HttpURLConnection) url.openConnection();
        if (userAgent != null) connection.setRequestProperty("User-Agent", userAgent);

        if (redirects >= 0) connection.setInstanceFollowRedirects(false);

        connection.setDoOutput(true);

        try (var out = new PrintWriter(connection.getOutputStream())) {
            boolean first = true;
            for (Map.Entry<Object, Object> pair : nameValuePairs.entrySet()) {
                if (first)
                    first = false;
                else
                    out.print("&");
                String name = pair.getKey().toString();
                String value = pair.getValue().toString();
                out.print(name);
                out.print("=");
                out.print(URLEncoder.encode(value, StandardCharsets.UTF_8));
            }
        }
        String encoding = connection.getContentEncoding();
        if (encoding == null) encoding = "UTF-8";

        if (redirects > 0) {
            int responseCode = connection.getResponseCode();
            if (responseCode == HttpURLConnection.HTTP_MOVED_PERM
                    || responseCode == HttpURLConnection.HTTP_MOVED_TEMP
                    || responseCode == HttpURLConnection.HTTP_SEE_OTHER) {
                String location = connection.getHeaderField("Location");
                if (location != null) {
                    URL base = connection.getURL();
                    connection.disconnect();
                    return doPost(base.toURI().resolve(location).toURL(), nameValuePairs,
                            userAgent, redirects - 1);
                }
            }
        }
        else if (redirects == 0) { throw new IOException("Too many redirects"); }

        var response = new StringBuilder();
        try (var in = new Scanner(connection.getInputStream(), encoding)) {
            while (in.hasNextLine()) {
```

```
                response.append(in.nextLine());
                response.append("\n");
            }
        }
        catch (IOException e) {
            InputStream err = connection.getErrorStream();
            if (err == null) throw e;
            try (var in = new Scanner(err)) {
                response.append(in.nextLine());
                response.append("\n");
            }
        }

        return response.toString();
    }
}
```

`java.net.HttpURLConnection` 1.0

- `InputStream getErrorStream()`
  returns a stream from which you can read server error messages.

`java.net.URLEncoder` 1.0

- `static String encode(String s, String encoding)` 1.4
  returns the URL-encoded form of the string `s`, using the given character encoding scheme. (The recommended scheme is `"UTF-8"`.) In URL encoding, the characters `A`–`Z`, `a`–`z`, `0`–`9`, `- _ . ~` are left unchanged. Space is encoded into `+`, and all other characters are encoded into sequences of encoded bytes of the form `%XY`, where `0xXY` is the hexadecimal value of the byte.

`java.net.URLDecoder` 1.2

- `static string decode(String s, String encoding)` 1.4
  returns the decoding of the URL encoded string `s` under the given character encoding scheme.

4.4. The HTTP Client

The `URLConnection` class was designed before HTTP was the universal protocol of the Web. It provides support for a number of protocols, but its HTTP support is somewhat cumbersome. When the decision was made to support HTTP/2, it became clear that it would be best to provide a modern client interface instead of reworking the existing API. The `HttpClient` provides HTTP/2 support and a more convenient API than the `URLConnection` class with its rather fussy set of stages.

4.4.1. The `HttpClient` Class

An `HttpClient` can issue requests and receive responses. You get a client by calling

```
HttpClient client = HttpClient.newHttpClient()
```

Alternatively, if you need to configure the client, you use a builder API, like this:

```
HttpClient client = HttpClient.newBuilder()
    .followRedirects(HttpClient.Redirect.ALWAYS)
    .build();
```

That is, you get a builder, call methods to customize the item that is going to be built, and then call the `build` method to finalize the building process. This is a common pattern for constructing immutable objects.

There are a number of advanced options for selecting HTTP 2 and for controlling cookies, SSL, proxy settings, and authentication.

The `HttpClient` is autocloseable, so you can declare it in a `try`-with-resources statement. Its `close` method waits for the completion of submitted requests and then closes its connection pool.

4.4.2. The `HttpRequest` class and Body Publishers

You also follow the builder pattern for formulating requests. Here is a `GET` request:

```
HttpRequest request = HttpRequest.newBuilder()
    .uri(new URI(urlString))
    .GET()
    .build();
```

The URI is the "uniform resource identifier" which is, when using HTTP, the same as a URL. However, in Java, the `URL` class has methods for actually opening a connection to a URL, whereas the `URI` class is only concerned with the syntax (scheme, host, port, path, query, fragment, and so on).

With a `POST` request, you need a "body publisher" that turns the request data into the data that is being posted. There are body publishers for strings, byte arrays, and files. For example, if your request is in JSON, you just provide the JSON string to a string body publisher.

```
HttpRequest request = HttpRequest.newBuilder()
    .uri(new URI(urlString))
    .header("Content-Type", "application/json")
    .POST(HttpRequest.BodyPublishers.ofString(jsonString))
    .build();
```

It is unfortunate that the API does not support the required formatting for common content types. The sample program in **Listing 4.10** provides body publishers for form data and file uploads.

In that program, you can see how body publishers are used to send strings and files. The file upload requires both, employing the `BodyPublishers.concat` method to concatenate multiple body publishers to produce the request body.

The `HttpRequest.Builder` class also has build methods for the less common `PUT`, `DELETE`, and `HEAD` requests.

Java 16 adds a builder for filtering the headers of an existing `HttpRequest`. You provide the request and a function that receives the header names and values, returning `true` for those that should be retained. For example, here we modify the content type:

```
HttpRequest request2 = HttpRequest.newBuilder(request, (name, value) ->
        !name.equalsIgnoreCase("Content-Type")) // Remove old content type
    .header("Content-Type", "application/xml") // Add new content type
    .build();
```

4.4.3. The `HttpResponse` Interface and Body Handlers

When sending the request, you have to tell the client how to handle the response. If you just want the body as a string, send the request with a `HttpResponse.BodyHandlers.ofString()`, like this:

```
HttpResponse<String> response
    = client.send(request, HttpResponse.BodyHandlers.ofString());
```

`HttpResponse` is a generic interface whose type parameter denotes the type of the response body. You get the response body string simply as

```
String bodyString = response.body();
```

There are other response body handlers that get the response as a byte array or input stream. `BodyHandlers.ofFile(filePath)` yields a handler that saves the response to the given file, and `BodyHandlers.ofFileDownload(directoryPath)` saves the response in the given directory, using the file name from the `Content-Disposition` header. Finally, the handler obtained from `BodyHandlers.discarding()` simply discards the response.

Processing the contents of the response is not considered part of the API. For example, if you receive JSON data, you need a JSON library to parse the contents.

The `HttpResponse` object also yields the status code and the response headers.

```
int status = response.statusCode();
HttpHeaders responseHeaders = response.headers();
```

You can turn the `HttpHeaders` object into a map:

```
Map<String, List<String>> headerMap = responseHeaders.map();
```

The map values are lists since in HTTP, each key can have multiple values.

If you just want the value of a particular key, and you know that there won't be multiple values, call the `firstValue` method:

```
Optional<String> lastModified = responseHeaders.firstValue("Last-Modified");
```

You get the response value or an empty optional if none was supplied.



Note: The `HttpHeaders` class is aware that keys for HTTP parameters are case-insensitive. If you call `responseHeaders.firstValue("Last-Modified")`, you get the desired header value even if it was sent

as `last-modified`.

4.4.4. Asynchronous Processing

You can process the response asynchronously. When building the client, provide an executor:

```
ExecutorService executor = Executors.newCachedThreadPool();
HttpClient client = HttpClient.newBuilder().executor(executor).build();
```

Build a request and then invoke the `sendAsync` method on the client. You receive a `CompletableFuture<HttpResponse<T>>`, where `T` is the type of the body handler. Use the `CompletableFuture` API as described in **Chapter 10 of Volume I**:

```
HttpRequest request = HttpRequest.newBuilder().uri(uri).GET().build();
client.sendAsync(request, HttpResponse.BodyHandlers.ofString())
    thenAccept(response -> . . .);
```



Tip: To enable logging for the `HttpClient`, add this line to `net.properties` in your JDK:

```
jdk.httpclient.HttpClient.log=all
```

Instead of `all`, you can specify a comma-separated list of the following event types:

- `channel`
- `content`
- `errors`
- `frames`, optionally followed by one or more of `:control`, `:data`, `:window`, or `:all`
- `headers`
- `requests`
- `ssl`
- `trace`

Don't use any spaces:

```
jdk.httpclient.HttpClient.log=headers,requests,frames:control:data
```

Then, set the logging level for the logger with name `jdk.httpclient.HttpClient` to `INFO`, for example by adding this line to the `logging.properties` file in your JDK:

```
jdk.httpclient.HttpClient.level=INFO
```

The program in **Listing 4.10** shows how to make synchronous requests to different services. The class in **Listing 4.11** implements the body publishers for forms, file upload, and JSON. Run the program as:

```
javac v2ch04/httpClient/HttpClientDemo.java
java v2ch04.httpClient.HttpClientDemo v2ch04/httpClient/form.properties
java v2ch04.httpClient.HttpClientDemo v2ch04/httpClient/fileupload.properties
```

```
java v2ch04.httpClient.HttpClientDemo v2ch04/httpClient/json.properties
```

With each run, a “Hello, World” program is sent to the CodeCheck service, and the result of running it is displayed. That service can accept form posts, file uploads, and JSON posts. Note the logging output from the `HttpClient`.

Listing 4.10 v2ch04/httpClient/HttpClientDemo.java

```
1  package v2ch04.httpClient;
2
3  import module java.base;
4  import module java.net.http;
5  import java.net.http.HttpRequest.BodyPublisher;
6
7  /**
8   * This program demonstrates the HTTP client
9   */
10 class HttpClientDemo {
11     void main(String[] args) throws Exception {
12         System.setProperty("jdk.httpclient.HttpClient.log", "headers,requests,content");
13         String propsFilename = args.length > 0 ? args[0] : "v2ch04/httpClient/json.properties";
14         Path propsPath = Path.of(propsFilename);
15         var props = new Properties();
16         try (Reader in = Files.newBufferedReader(propsPath)) {
17             props.load(in);
18         }
19         String urlString = "" + props.remove("url");
20         String contentType = "" + props.remove("Content-Type");
21         BodyPublisher publisher = null;
22         if (contentType.equals("application/x-www-form-urlencoded"))
23             publisher = MoreBodyPublishers.ofFormData(props);
24         else if (contentType.equals("multipart/form-data")) {
25             // Split each value along commas, replace strings starting with
26             // file:// with Path objects
27             var data = new HashMap<String, List<?>>();
28             for (Map.Entry<Object, Object> entry : props.entrySet()) {
29                 data.put(entry.getKey().toString(),
30                     Stream.of(entry.getValue().toString().split("\\s*,\\s*"))
31                         .map(s -> s.startsWith("file://")
32                                 ? propsPath.getParent().getParent()
33                                         .resolve(Path.of(s.substring(7)))
34                                 : s)
35                         .toList());
36             }
37             String boundary = UUID.randomUUID().toString().replace("-", "");
38             contentType += "; boundary=" + boundary;
39             publisher = MoreBodyPublishers.ofMimeMultipartData(data, boundary);
40         }
41         else {
42             contentType = "application/json";
43             publisher = MoreBodyPublishers.ofSimpleJSON(props);
44         }
45
46         String result = doPost(urlString, contentType, publisher);
47         System.out.println(result);
48     }
```

```
49
50      String doPost(String url, String contentType, BodyPublisher publisher)
51              throws IOException, URISyntaxException, InterruptedException {
52          try (HttpClient client = HttpClient.newBuilder()
53                  .followRedirects(HttpClient.Redirect.ALWAYS).build()) {
54
55              HttpRequest request = HttpRequest.newBuilder().uri(new URI(url))
56                      .header("Content-Type", contentType).POST(publisher).build();
57
58              HttpResponse<String> response = client.send(request,
59                      HttpResponse.BodyHandlers.ofString());
60              return response.body();
61          }
62      }
63  }
```

Listing 4.11 v2ch04/httpClient/MoreBodyPublishers.java

```
1   package v2ch04.httpClient;
2
3   import module java.base;
4   import java.net.http.HttpRequest.*;
5
6   public class MoreBodyPublishers {
7       public static BodyPublisher ofFormData(Map<Object, Object> data) {
8           boolean first = true;
9           var builder = new StringBuilder();
10          for (Map.Entry<Object, Object> entry : data.entrySet()) {
11              if (first)
12                  first = false;
13              else
14                  builder.append("&");
15              builder.append(
16                      URLEncoder.encode(entry.getKey().toString(), StandardCharsets.UTF_8));
17              builder.append("=");
18              builder.append(
19                      URLEncoder.encode(entry.getValue().toString(), StandardCharsets.UTF_8));
20          }
21          return BodyPublishers.ofString(builder.toString());
22      }
23
24      public static BodyPublisher ofMimeMultipartData(Map<String, List<?>> data, String boundary)
25              throws IOException {
26          var bps = new ArrayList<BodyPublisher>();
27          var header = new StringBuilder();
28          for (Map.Entry<String, List<?>> entry : data.entrySet()) {
29              for (Object value : entry.getValue()) {
30                  header.append("--%s\r\nContent-Disposition: form-data; name=%s"
31                          .formatted(boundary, entry.getKey()));
32
33                  if (value instanceof Path path) {
34                      header.append("; filename=\"%s\"\r\n".formatted(path.getFileName()));
35                      String mimeType = Files.probeContentType(path);
36                      if (mimeType != null)
37                          header.append("Content-Type: %s\r\n".formatted(mimeType));
38                      header.append("\r\n");
39                      bps.add(BodyPublishers.ofString(header.toString()));
```

```
                    bps.add(BodyPublishers.ofFile(path));
                }
                else {
                    header.append("\r\n\r\n%s\r\n".formatted(value));
                    header.append("\r\n");
                    bps.add(BodyPublishers.ofString(header.toString()));
                }
                header = new StringBuilder("\r\n");
            }
        }
        bps.add(BodyPublishers.ofString("\r\n--%s--\r\n".formatted(boundary)));
        return BodyPublishers.concat(bps.toArray(BodyPublisher[]::new));
    }

    public static BodyPublisher ofSimpleJSON(Map<Object, Object> data) {
        var builder = new StringBuilder();
        builder.append("{");
        var first = true;
        for (Map.Entry<Object, Object> entry : data.entrySet()) {
            if (first)
                first = false;
            else
                builder.append(",");
            builder.append(jsonEscape(entry.getKey().toString())).append(": ")
                    .append(jsonEscape(entry.getValue().toString()));
        }
        builder.append("}");
        return BodyPublishers.ofString(builder.toString());
    }

    private static Map<Character, String> replacements = Map.of('\b', "\\b", '\f', "\\f", '\n',
            "\\n", '\r', "\\r", '\t', "\\t", '"', "\\\"", '\\', "\\\\");

    private static StringBuilder jsonEscape(String str) {
        var result = new StringBuilder("\"");
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            String replacement = replacements.get(ch);
            if (replacement == null)
                result.append(ch);
            else
                result.append(replacement);
        }
        result.append("\"");
        return result;
    }
}
```

`java.net.http.HttpClient` 11

- `static HttpClient newHttpClient()`
  yields an `HttpClient` with a default configuration.
- `static HttpClient.Builder newBuilder()`
  yields a builder for building an `HttpClient`.

- `<T> HttpResponse<T> send(HttpRequest request, HttpResponse.BodyHandler<T> responseBodyHandler)`
- `<T> CompletableFuture<HttpResponse<T>> sendAsync(HttpRequest request, HttpResponse.BodyHandler<T> responseBodyHandler)`
  make a synchronous or asynchronous request and process the response body with the given handler.

`java.net.http.HttpClient.Builder` 11

- `HttpClient build()`
  yields an `HttpClient` with the properties configured by this builder.
- `HttpClient.Builder followRedirects(HttpClient.Redirect policy)`
  sets the redirect policy to one of the values `ALWAYS`, `NEVER`, or `NORMAL` (only refuse redirects from HTTPS to HTTP) of the `HttpClient.Redirect` enumeration.
- `HttpClient.Builder executor(Executor executor)`
  sets the executor for asynchronous requests.

`java.net.http.HttpRequest` 11

- `HttpRequest.Builder newBuilder()`
  returns a builder for building an `HttpRequest`.
- `HttpRequest.Builder newBuilder(HttpRequest request, BiPredicate<String,String> filter)` 16
  returns a builder for building a request with the same properties as the provided one, with only the headers that are accepted by the filter.

`java.net.http.HttpRequest.Builder` 11

- `HttpRequest build()`
  yields an `HttpRequest` with the properties configured by this builder.
- `HttpRequest.Builder uri(URI uri)`
  sets the URI for this request.
- `HttpRequest.Builder header(String name, String value)`
  sets a request header for this request.
- `HttpRequest.Builder GET()`
- `HttpRequest.Builder DELETE()`
- `HttpRequest.Builder POST(HttpRequest.BodyPublisher bodyPublisher)`
- `HttpRequest.Builder PUT(HttpRequest.BodyPublisher bodyPublisher)`
  set the request method and body for this request.

`java.net.http.HttpResponse<T>` 11

- `T body()`
  yields the body of this response.
- `int statusCode()`
  yields the status code for this response.
- `HttpHeaders headers()`
  yields the response headers.

`java.net.http.HttpHeaders` 11

- `Map<String,List<String>> map()`
  yields a `Map` of these headers.
- `Optional<String> firstValue(String name)`
  yields the first value with the given name in these headers, if present.

4.5. The Simple HTTP Server

Since Java 6, the JDK has provided a simple web server in the `com.sun.net.httpserver` package. In Java 18, a command-line tool was added to the JDK that invokes a static file server based upon that package. At that time, a few convenience classes were added, and JEP 408 affirms that the package is "officially supported." The web server is intended for testing, development, and debugging.

4.5.1. The Command-Line Tool

To start the simple file server, use the command

```
jwebserver
```

It serves all files in the current directory and its subdirectories on port 8000. When a file is served, a message is printed. You can change the directory and port with the `-d` and `-p` options:

```
jwebserver -d /tmp -p 8189
```

You can also set the message output level with the `-o` command-line option. Choices are `info` (the default), `verbose`, and `none`.

To kill the server, press Ctrl+C or use the `kill`/`pkill` command.

This server is only useful for serving files. It is similar to the `python -m http.server` command. In fact, you can also invoke the simple Java file server as `java -m jdk.httpserver`.

4.5.2. The HTTP Server API

You can start the simple file server programmatically as:

```
import com.sun.net.httpserver.*;
. . .
HttpServer server = SimpleFileServer.createFileServer(new InetSocketAddress(port),
    Path.of("/tmp"), OutputLevel.VERBOSE);
server.start();
```

Now you can customize it in various ways. Alternatively, if you don't want to serve files, you can get a do-nothing server as

```
HttpServer server = HttpServer.create(new InetSocketAddress(port), 0);
```

The second argument is the "backlog" of the server socket—that is, the maximum number of queued connection requests. With a zero argument, a default value is used.

You can set the executor for processing requests:

```
server.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
```

For the server to do anything interesting, you need to create *contexts*.

A context has

- A *path* that must be a prefix of the request path
- A *handler* that computes the response
- A list of *filters* that can transform the request or response (see **Section 4.5.4**)

If a request matches multiple contexts, the one with the longest path is executed. For example, here we install two handlers for paths `/static/` and `/`:

```
server.createContext("/static/", SimpleFileHandler.createFileHandler(staticRoot));
server.createContext("/", HttpHandlers.of(301,
    Headers.of("Location", "https://horstmann.com/corejava"), ""));
```

Note that you can leverage existing handlers, such as the handler of the simple file server.

The static `HttpHandlers.of` method produces a simple handler that returns a fixed response. You pass the status code, response headers, and response body.

The static `HttpHandlers.handleOrElse` uses a `Predicate<Request>` to select among two handlers:

```
HttpHandler handler = HttpHanders.handleOrElse(
    request -> request.getRequestMethod().equals("GET"),
    SimpleFileHandler.createFileHandler(staticRoot).
    HttpHandlers.of(500, Headers.of(), "Bad request method"))
```

For more complex handlers, you need to provide an instance of a subclass of `HttpHandler`. See the next section for details.

4.5.3. Handlers

A handler produces the response to an HTTP request in its method

```
public void handle(HttpExchange exchange)
```

The `HttpExchange` instance holds both the request and response data. The handler must use the object in this specific order:

1. Read the request method, URI, and headers:

```
String method = exchange.getRequestMethod();
URI uri = exchange.getRequestURI();
Headers requestHeaders = exchange.getRequestHeaders();
String userAgent = requestHeaders.getFirst("user-agent");
```

2. Read the request body for `POST`, `PUT`, and `PATCH` requests:

```
InputStream in = exchange.getRequestBody();
String body = new String(in.readAllBytes());
in.close();
```

3. Set the response headers:

```
Headers responseHeaders = exchange.getResponseHeaders();
responseHeaders.add("Content-type", "text/plain");
. . .
```

4. Set the status code and the length of the response body:

```
exchange.sendResponseHeaders(200, responseBytes.length);
```

5. Write the response body, if there is one:

```
OutputStream responseBody = exchange.getResponseBody();
responseBody.write(responseBytes);
responseBody.close();
```

The request and response headers are instances of the `Headers` class, which implements `Map<String, List<String>>`. Recall that there can be multiple values for each header.

The `Headers` class is similar to the `HttpHeaders` that you saw in **Section 4.4**, but it directly implements the `Map<String, List<String>>` interface. As with `HttpHeaders`, the keys are case-insensitive.

If you know that there is at most one value for a given key, call `getFirst` to retrieve it. To add a key/value pair to a response map, use the `add` method, as in the code snippet above.

The sample program in **Listing 4.12** has an example of a handler that produces a text response with all details of the request.

4.5.4. Filters

In addition to a handler, a context can have a chain of filters. A filter can modify the request or response, or carry out additional work such as logging or authentication. For example, the simple web server uses a filter for displaying the requests.

To implement a filter, extend the abstract `Filter` class and implement two methods. The `description` method should return some descriptive string. Typically, the `doFilter` method looks like this:

```
public void doFilter(HttpExchange exchange, Filter.Chain chain) {
    Request processing
    chain.doFilter(exchange);
    Response processing
}
```

By calling `chain.doFilter(exchange)`, the remaining filters and the handler are invoked.

A filter can abort further filtering and handling—for example, if authentication fails. Then it should not call `chain.doFilter(exchange)`, and instead produce a response in the `exchange` object.

You add a filter like this:

```
context.getFilters().add(filter);
```



Caution: As you can see, the HTTP server API is a bit rough and old-fashioned. In your own code, do not provide methods that yield references to internal mutable data structures, as the `Context.getFilters` method does.

The `Filter` class has three static factory methods for filters:

```
public Filter beforeHandler(String description, Consumer<HttpExchange> operation)
public Filter afterHandler(String description, Consumer<HttpExchange> operation)
public Filter adaptRequest(String description, UnaryOperator<Request> requestOperator)
```

The first two pass the `exchange` object (either before or after the request) to the consumer that can inspect or modify it.

The third method makes it a bit more convenient to add a header to a request:

```
Filter f = Filter.adaptRequest("Adding header",
    request -> request.with(headerName, List.of(headerValue)));
```

Of course, compared with mature web servers, such as Tomcat or Jetty, this server is rather basic. But it is a part of the JDK and can be sufficient for prototyping and testing.

The program in **Listing 4.12** shows a simple server that echoes all request data in a plain-text response. Start it and point your browser to `http://localhost:8189` to see the request headers of your browser. The server is also useful for observing the form post and the file upload requests of the program in **Listing 4.10**.

Listing 4.12 v2ch04/httpServer/EchoServer.java

```
1  package v2ch04.httpServer;
2
3  import module java.base;
4  import module jdk.httpserver;
5
6  /**
7   * This program implements an HTTP server that listens to port 8189 or a port given as
```

```
 * argument, and echoes back all client input.
 */
class EchoServer {
    void main(String[] args) throws Exception {
        int port = args.length >= 1 ? Integer.parseInt(args[0]) : 8189;
        HttpServer server = HttpServer.create(new InetSocketAddress(port), 0);
        server.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
        server.createContext("/", HttpHandlers.of(301,
                Headers.of("Location", "https://horstmann.com/corejava"), ""));
        server.createContext("/echo", (HttpExchange exchange) -> {
            var headers = new StringBuilder();
            exchange.getRequestHeaders().forEach(
                    (k, vs) -> vs.forEach(v -> headers.append("%s: %s\n".formatted(k, v))));

            String requestBody = new String(exchange.getRequestBody().readAllBytes());
            String response = "%s %s\n%s\n%s\n".formatted(exchange.getRequestMethod(),
                    exchange.getRequestURI(), headers, requestBody);
            byte[] responseBytes = response.getBytes();

            exchange.sendResponseHeaders(200, responseBytes.length);
            OutputStream responseBody = exchange.getResponseBody();
            responseBody.write(responseBytes);
            responseBody.close();
        });
        server.start();
    }
}
```

4.6. Sending E-Mail

In the past, it was simple to write a program that sends e-mail by making a socket connection to port 25, the SMTP port, of a mail server. The Simple Mail Transport Protocol (SMTP) describes the format for e-mail messages. Once you are connected to the server, send a mail header (in the SMTP format, which is easy to generate), followed by the mail message.

Here are the details:

1. Open a socket to your host.

```
var s = new Socket("mail.yourserver.com", 25); // 25 is SMTP
var out = new PrintWriter(s.getOutputStream());
```

2. Send the following information to the print stream:

```
HELO sending host
MAIL FROM: sender e-mail address
RCPT TO: recipient e-mail address
DATA
Subject: subject
(blank line)
mail message (any number of lines)
.
QUIT
```

The SMTP specification (RFC 821) states that lines must be terminated with `\r` followed by `\n`.

It used to be that SMTP servers were often willing to route e-mail from anyone. However, in these days of spam floods, most servers have built-in checks and only accept requests from users or IP address ranges that they trust. Authentication usually happens over secure socket connections.

Implementing these authentication schemes manually would be very tedious. Instead, we will show you how to use the Jakarta Mail API to send e-mail from a Java program.

To use Jakarta Mail, you need to set up some properties that depend on your mail server. For example, with GMail, you use

```
mail.transport.protocol=smtps
mail.smtps.auth=true
mail.smtps.host=smtp.gmail.com
mail.smtps.user=accountname@gmail.com
```

Our sample program reads these from a property file.

For security reasons, we don't put the password into the property file but instead prompt for it.



Caution: With most mail providers, you need to generate and use a special application password instead of using your normal mail password.

Read in the property file, then get a mail session like this:

```
Session mailSession = Session.getDefaultInstance(props);
```

Make a message with the desired sender, recipient, subject, and message text:

```
var message = new MimeMessage(mailSession);
message.setFrom(new InternetAddress(from));
message.addRecipient(RecipientType.TO, new InternetAddress(to));
message.setSubject(subject);
message.setText(builder.toString());
```

Then send it off:

```
Transport tr = mailSession.getTransport();
tr.connect(null, password);
tr.sendMessage(message, message.getAllRecipients());
tr.close();
```

The program reads the message from a text file of the format

```
Sender
Recipient
Subject
Message text (any number of lines)
```

At the time of this writing, GMail does not check the veracity of the information—you can supply any sender you like. (Keep this in mind the next time you get an e-mail message from `president@whitehouse.gov` inviting you to a black-tie affair on the front lawn.)

Because the program requires external JAR files, it is not contained in the `corejava-bookcode.zip` file, but instead in `corejava-modulechapter.zip`. You don't need to use modules to run the program. Simply compile and run as follows:

```
cd corejava-modulechapter/v2ch04.mail
javac -classpath .:../jars/\* com/horstmann/mail/MailDemo.java
java -classpath .:../jars/\* com.horstmann.mail.MailDemo path/to/message.txt
```

Or, if you prefer, use modules:

```
cd corejava-modulechapter/
javac -p v2ch04.mail:jars/jakarta.mail-api-2.1.3.jar:jars/jakarta.activation-api-2.1.3.jar \
    v2ch04.mail/com/horstmann/mail/MailDemo.java v2ch04.mail/module-info.java
java -p v2ch04.mail:jars/jakarta.mail-api-2.1.3.jar:jars/jakarta.activation-api-2.1.3.jar\
:jars/angus-activation-2.0.2.jar:jars/angus-mail-2.0.3.jar \
    -m v2ch04.mail/com.horstmann.mail.MailDemo path/to/message.txt
```



Tip: If you can't figure out why your mail connection isn't working, call

```
mailSession.setDebug(true);
```

and check out the messages. Also, the Jakarta Mail API FAQ at **https://eclipse-ee4j.github.io/angus-mail/FAQ** has some useful debugging hints.

In this chapter, you have seen how to write network clients and servers in Java and how to harvest information from web servers. The next chapter covers database connectivity. You will learn how to work with relational databases in Java, using the JDBC API.

Previous chapter
< 3. XML

Next chapter
5. Database Programming >